

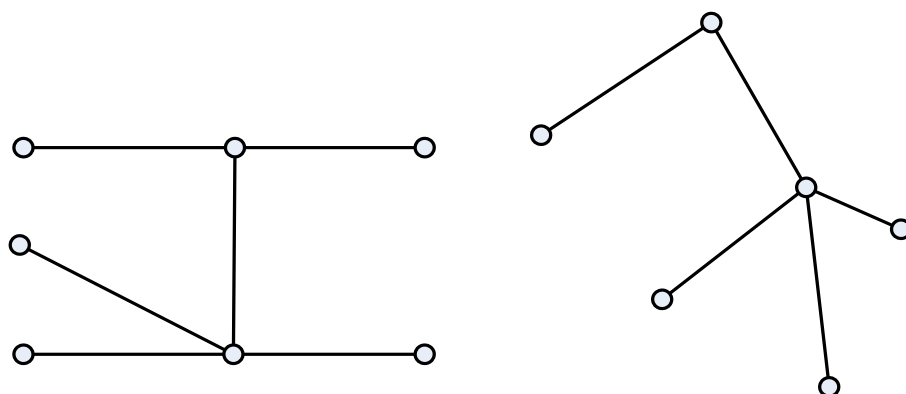
CHƯƠNG 8. CÂY

Chương này trình bày một dạng đồ thị đặc biệt, có nhiều ứng dụng trong tin học, đó là cây. Người đầu tiên sử dụng khái niệm cây để biểu diễn là nhà toán học người Anh tên là Arthur Cayley. Năm 1857, ông dùng cây để xác định những dạng khác nhau của hợp chất hóa học. Sau đây sẽ trình bày các khái niệm, tính chất và những thuật toán duyệt cây.

8.1. ĐỊNH NGHĨA VÀ CÁC TÍNH CHẤT CƠ BẢN CỦA CÂY

Cây là một đồ thị vô hướng, liên thông, và không có chu trình.

Ví dụ:



Hình 8.1: Cây

Rừng là một đồ thị vô hướng và không có chu trình. Như vậy, rừng có thể có nhiều thành phần liên thông và mỗi thành phần liên thông là một cây.

➤ Các tính chất cơ bản của cây

Các tính chất cơ bản của cây thể hiện trong định lý sau:

✓ Định lý nhận biết cây

Cho $T = (V, E)$ là đồ thị vô hướng n đỉnh. Các mệnh đề sau đây là tương đương:

MĐ1: T là cây (T liên thông và không chứa chu trình).

MĐ2: T không chứa chu trình và có $n-1$ cạnh.

MĐ3: T liên thông và có $n-1$ cạnh.

MĐ4: T liên thông và mỗi cạnh của nó đều là cầu.

MĐ5: Hai đỉnh bất kỳ của T được nối với nhau bởi đúng 1 đường đi đơn.

MĐ6: T không chứa chu trình nhưng nếu thêm vào nó một cạnh ta thu được đúng 1 chu trình.

Chứng minh:

Ta sẽ chứng minh định lý trên theo sơ đồ sau:

$$\text{MĐ1} \Rightarrow \text{MĐ2} \Rightarrow \text{MĐ3} \Rightarrow \text{MĐ4} \Rightarrow \text{MĐ5} \Rightarrow \text{MĐ6} \Rightarrow \text{MĐ1}$$

Chứng minh MĐ1 $\Rightarrow$ MĐ2: Nếu T là cây n đỉnh thì T không có chu trình và có $n-1$ cạnh

T không có chu trình là hiển nhiên, do đó, chỉ cần chứng minh rằng T có $n-1$ cạnh

Ta chứng minh bằng phương pháp quy nạp theo số đỉnh của cây

+ Với $n=1$ thì đồ thị có số cạnh là: $n-1 = 1 - 1 = 0$ (Đúng)

+ Giả sử khẳng định đúng $\forall$ cây có $k \geq 1$ đỉnh.

T là cây có $k+1$ đỉnh. Cần phải chỉ ra T có k cạnh.

Chọn đường đi dài nhất trong T là $P = (v_1, v_2, \dots, v_m)$. Rõ ràng v_1 là đỉnh treo vì:

+ v_1 không thể kề với các đỉnh $v_3, \dots, v_m$ vì T không có chu trình

+ v_1 không thể được nối với các đỉnh khác vì P là đường đi dài nhất dài nhất

Xét cây T' có được từ T sau khi bỏ đi đỉnh v_1 và cạnh (v_1, v_2) . Rõ ràng T' cũng là cây và có k đỉnh, và theo giả thiết quy nạp T' có $k-1$ cạnh $\Rightarrow T$ có k cạnh (ĐPCM).

Chứng minh MĐ2 $\Rightarrow$ MĐ3: Nếu T không chứa chu trình và có $n-1$ cạnh thì T liên thông

Ta chứng minh bằng phương pháp phản chứng. Giả sử T không liên thông, khi đó T được phân rã thành $k > 1$ thành phần liên thông $T_1, T_2, \dots, T_k$. Vì T không chứa chu trình (theo giả thiết) nên các T_i cũng vậy, suy ra T_i là cây.

Gọi $v(T)$ và $e(T)$ tương ứng là số đỉnh và cạnh của T . Theo phần trước MĐ1 $\Rightarrow$ MĐ2 ta có: $e(T_i) = v(T_i) - 1$. Suy ra:

$$\sum e(T_i) = \sum (v(T_i) - 1) = \sum v(T_i) - k$$

$$\Leftrightarrow e(T) = v(T) - k \text{ Hay } n - 1 = n - k. \text{ Vô lý với } k > 1 \text{ (ĐPCM)}$$

Chứng minh MĐ3 $\Rightarrow$ MĐ4: Nếu T liên thông và có $n-1$ cạnh thì mỗi cạnh của T là cầu.

Xóa 1 cạnh bất kỳ của T , khi đó ta nhận được cây T' có n đỉnh và $n-2$ cạnh. Nếu ta chỉ ra được T' không liên thông thì cạnh đó là cầu của T

Ta sẽ chứng minh rằng đồ thị với n đỉnh và $n-2$ cạnh sẽ không liên thông. Chứng minh bằng Quy nạp theo n và phản chứng giống MĐ1 $\Rightarrow$ MĐ2. Chọn đường đi dài nhất và xóa bỏ cạnh treo cùng đỉnh (ĐPCM)

Chứng minh MĐ4 $\Rightarrow$ MĐ5: Nếu T liên thông và mỗi cạnh của T là cầu thì hai đỉnh bất kỳ của T được nối với nhau đúng bởi 1 đường đơn.

T liên thông nên mọi cặp đỉnh của T tồn tại đường nối giữa chúng. Đường nối này là duy nhất vì trái lại T sẽ có chu trình và các cạnh trên chu trình đó sẽ không thể là cầu. (ĐPCM)

Chứng minh MĐ5 $\Rightarrow$ MĐ6: Nếu hai đỉnh bất kỳ của T được nối với nhau đúng bởi 1 đường đơn thì T không chứa chu trình, nhưng nếu thêm vào nó 1 cạnh ta thu được đúng 1 chu trình

+ T không chứa chu trình vì nếu T có chu trình thì sẽ có cặp đỉnh được nối với nhau bởi ít nhất 2 đường đơn.

+ Giả sử cạnh (u,v) được thêm vào ta sẽ nhận được chu trình gồm đường đơn nối u với v và cạnh (u,v) mới.

+ Do đường đơn nối trên là duy nhất nên chu trình nhận được cũng là duy nhất. (ĐPCM)

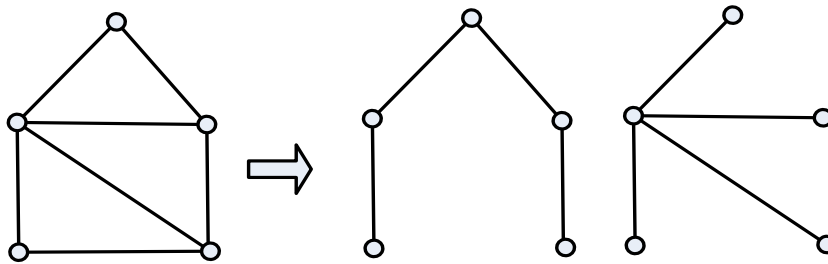
8.2. CÂY KHUNG CỦA ĐỒ THỊ

8.2.1. Định nghĩa

Cho đồ thị $G=(V, E)$ vô hướng, liên thông. Một cây $T=(V, F)$ được xây dựng từ G với $F \subset E$ (T chứa tất cả các đỉnh của G và tập cạnh F là con của tập cạnh E) được gọi là **cây khung** của đồ thị G .

Người ta còn gọi cây khung là: cây bao trùm hay cây tối đại.

Ví dụ:



Hình 8.2: Cây khung

8.2.2. Thuật toán xây dựng cây khung

Phần này trình bày phương pháp xây dựng cây khung dựa trên các thuật toán duyệt đồ thị theo chiều sâu, và theo chiều rộng.

Đầu vào của các thuật toán: Đồ thị G lưu dưới dạng danh sách kề - Mảng $Ke[]$

Kết quả trả về: Cây khung T của đồ thị

Mảng $ChuaXet[]$ dùng để đánh dấu các đỉnh đã được xét hay chưa.

a. Xây dựng cây khung theo chiều sâu

```
/* Khai báo các biến toàn cục ChuaXet, Ke, T */
```

```
void Tree_DFS(v);
```

```
{
```

```
    ChuaXet[v] = 0;
```

```
    for (u ∈ Ke(v))
```

```
        if (ChuaXet[u])
```

```
        {
```

```

        T = T ∪ (v,u);
        Tree_DFS(u);
    };

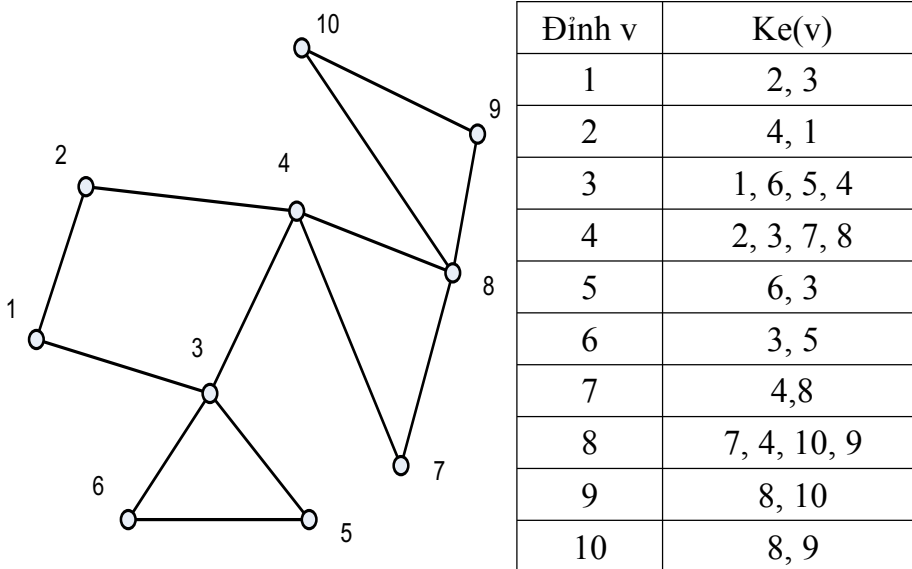
}

main()
{
    /* Nhập đồ thị, tạo biến Ke */
    for (v ∈ V)
        ChưaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
    T = ∅; /* T là tập cạnh cây khung */
    Tree_DFS(root); /* root là đỉnh nào đó của đồ thị */
}

```

Ví dụ:

Áp dụng giải thuật trên, tìm cây khung cho đồ thị G sau



Hình 8.3: Tìm cây khung theo chiều sâu

Áp dụng thuật toán xây dựng cây khung theo chiều sâu, bắt đầu từ đỉnh 1, thứ tự các đỉnh được duyệt như sau:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 5$
 $7 \rightarrow 8 \rightarrow 10 \rightarrow 9$

Khi đó cây khung của G là: $\{(1, 2), (2, 4), (4, 3), (3, 6), (6, 5), (4, 7), (7, 8), (8, 10), (10, 9)\}$

b. Xây dựng cây khung theo chiều rộng

```

/* Khai báo các biến toàn cục ChuaXet, Ke, QUEUE */
void Tree_BFS(r);
{
    QUEUE =  $\emptyset$ ;
    QUEUE  $\leftarrow$  r;
    ChuaXet[r] = 0;
    while (QUEUE  $\neq \emptyset$ )
    {
        v  $\leftarrow$  QUEUE;
        for (u  $\in$  Ke(v))
            if (ChuaXet[u])
            {
                QUEUE  $\leftarrow$  u;
                ChuaXet[u] = 0;
                T = T  $\cup$  (v,u);
            };
    }
}
main() /* Nhập đồ thị, tạo biến Ke */
{
    for (v  $\in$  V)
        ChuaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
    T =  $\emptyset$ ; /* T là tập cạnh cây khung */
    Tree_DFS(root); /* root là đỉnh nào đó của đồ thị */
}

```

Ví dụ:

Áp dụng thuật toán xây dựng cây khung theo chiều rộng cho đồ thị (Hình 5.3).

Bắt đầu duyệt từ đỉnh 1. Thứ tự duyệt các đỉnh như sau:

1 $\rightarrow$ 2 $\rightarrow$ 3

4 $\rightarrow$ 6 $\rightarrow$ 5

7 $\rightarrow$ 8

10 $\rightarrow$ 9

Khi đó, cây khung của G là: $\{(1, 2), (2, 3), (2, 4), (4, 6), (6, 5), (4, 7), (7, 8), (8, 10), (10, 9)\}$

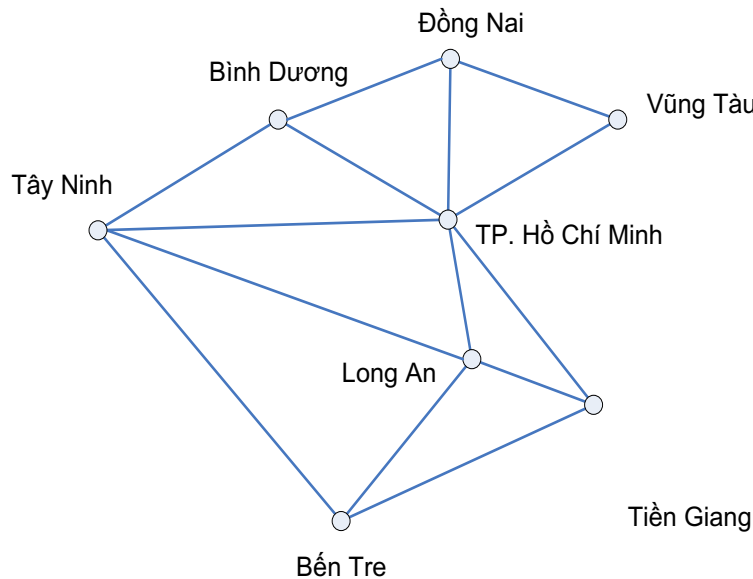
8.3. BÀI TOÁN CÂY KHUNG NHỎ NHẤT

Ở các phần trên, chúng ta đã đề cập đến khái niệm cây khung và các thuật toán tìm cây khung cho một đồ thị vô hướng, liên thông. Trong phần này, chúng ta sẽ nghiên cứu các thuật toán tìm cây khung mà có tổng trọng số các cạnh là nhỏ nhất. Trước hết cần nắm một số khái niệm.

Cho đồ thị G vô hướng, liên thông, có trọng số. Giả sử $T = (V, F)$ là cây khung của G. Tổng trọng số các cạnh của T (hay được gọi là trọng số của cây T) được định nghĩa: $w(T) = \sum_{e \in F} w(e)$.

Cây khung mà có trọng số nhỏ nhất trong số các cây khung của G được gọi là cây khung nhỏ nhất.

Bài toán tìm cây khung nhỏ nhất có nhiều ứng dụng trong việc giải quyết các vấn đề thực tế như: Bài toán xây dựng hệ thống đường sắt sao cho chi phí xây dựng nhỏ nhất nhưng vẫn bảo đảm nhu cầu giao lưu giữa các thành phố với nhau. Bài toán nối mạng máy tính sao cho chi phí lắp đặt là nhỏ nhất, v.v.



Hình 8.4

Xây dựng hệ thống giao thông để nối liền các vùng, sao cho tổng quãng đường xây dựng là ngắn nhất.

Sau đây là các thuật toán tìm cây khung nhỏ nhất thông dụng.

8.3.1. Thuật toán Kruskal

Nội dung của thuật toán

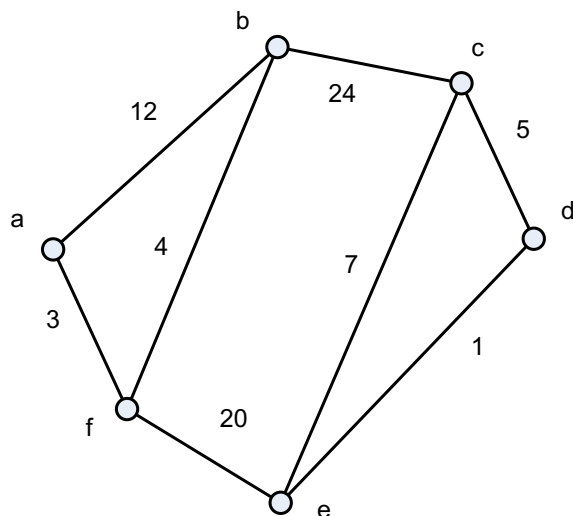
Xây dựng tập cạnh F của $T = (V, F)$ theo từng bước:

1. Sắp xếp các cạnh của G theo thứ tự trọng số (độ dài) tăng dần
2. Bắt đầu với $F = \emptyset$ bổ sung dần các cạnh của G vào F với điều kiện không tạo nên chu trình trong T.

3. Thuật toán dừng lại khi có n-1 cạnh được chọn.

Ví dụ:

Áp dụng thuật toán Kruskal, tìm cây khung nhỏ nhất cho đồ thị sau:



Hình 8.5: Ví dụ thuật toán Kruskal

Danh sách các cạnh theo thứ tự trọng số tăng dần:

(d, e)	(a, f)	(b, f)	(c, d)	(c, e)	(a, b)	(f, e)	(b, c)
1	3	4	5	7	12	20	24

Cây khung nhỏ nhất của đồ thị:

(d, e)	(a, f)	(b, f)	(c, d)	(f, e)
1	3	4	5	20

Sau đây là cài đặt thuật toán Kruskal bằng mã giả:

```
void Kruskal;
{
    F = ∅;
    while ((|F| < n-1) && (E != ∅))
    {
        Chọn e = min ∈ E;
        E = E \ {e};
        if (F ∪ {e} không chứa chu trình)
            F = F ∪ {e};
    }
}
```

```

    }
    if (|F| < n-1) cout << “Đồ thị không liên thông”;
}

```

Trong thực tế, để kiểm tra đồ thị không chứa chu trình không phải là đơn giản, mà trong thuật toán trên ta phải kiểm tra tính chất này trong mỗi vòng lặp. Do vậy, có thể tiếp cận bằng cách xây dựng “dần dần” cây khung nhỏ nhất theo cách sau:

- Đồ thị ban đầu của ta có n đỉnh và không có cạnh. Có thể coi đây như là một rừng gồm n cây
- Mỗi lần chúng ta sẽ thêm vào một cạnh theo thứ tự tăng dần theo độ dài sao cho hai đầu mút của cạnh mới được thêm phải nằm ở hai cây khác nhau
- Thuật toán sẽ dừng lại khi có $n-1$ cạnh được thêm

Áp dụng vào đồ thị trên hình 5.5 ta có:

Danh sách các cạnh theo thứ tự trọng số tăng dần:

(d, e)	(a, f)	(b, f)	(c, d)	(c, e)	(a, b)	(f, e)	(b, c)
1	3	4	5	7	12	20	24

Thực hiện theo các bước của thuật toán:

Chọn cạnh	CÁC CÂY					
	A	b	c	d	e	f
(d, e)=1	A	b	c	d,e		f
(a, f)=3	a,f	b	c	d,e		
(b, f)=4	a,f,b		c	d,e		
(c, d)=5	a,f,b		c,d,e			
(c, e)=7						

(a, b)=12						
(f, e)=20	a,f,b,c,d,e					
(b, c)=24						

Các cạnh đã được chọn cho cây khung nhỏ nhất:

(d, e)	(a, f)	(b, f)	(c, d)	(f, e)
1	3	4	5	20

Sau đây là cài đặt thuật toán Kruskal trên thực tế:

```

Kruskal()
{
    T = ∅;
    for (v ∈ V)
        MakeSet(v); //Mỗi đỉnh có một nhãn khác nhau
    xếp thứ tự các cạnh trong E tăng dần theo trọng số w
    for (u,v) ∈ E (theo thứ tự đã sắp xếp)
        if (FindSet(u) != FindSet(v)) // Kiểm tra hai đỉnh u và v có khác nhãn ?
            T = T ∪ { {u,v} }; //Chọn cạnh (u,v)
        Union(FindSet(u), FindSet(v)); //Đồng nhất nhãn của u và v
}

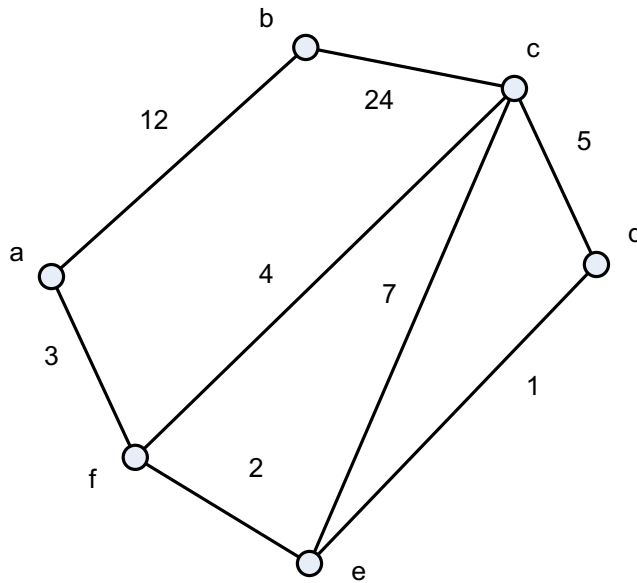
```

8.3.2. Thuật toán Prim

Nội dung của thuật toán như sau:

- Xây dựng tập đỉnh V_T và tập cạnh F của cây khung $T=(V_T, F)$ theo từng bước:
1. Bắt đầu với $V_T = \{s\}$, một đỉnh bất kỳ và $F=\emptyset$.
 2. Trong các cạnh có 1 đỉnh $\notin V_T$ và 1 đỉnh $\in V_T$ chọn cạnh có trọng số nhỏ nhất.
 3. Bổ sung cạnh đó vào F và đỉnh tương ứng vào V_T
 4. Thuật toán dừng lại khi có n-1 cạnh được chọn (hoặc $V_T = V$)

Ví dụ: Áp dụng thuật toán Prim, tìm cây khung nhỏ nhất cho đồ thị sau:



Hình 8.6: Thuật toán Prim

Để hiện thực thuật toán ta sẽ gán nhãn các đỉnh: Mỗi đỉnh $v \notin V_T$ sẽ có hai nhãn

- $d[v] = \min \{ w(v,u); u \in V_T \}$
- $\text{near}[v] = \{ u; \text{với } w(v,u) \text{ min} \}$

Thuật toán Prim sẽ được thực hiện theo các bước sau (Đỉnh bắt đầu là a):

V_T	b	c	d	e	f
a	12, a	∞ , a	∞ , a	∞ , a	3, a
a, f	12, a	4, f	∞ , a	2, f	
a, f, e	12, a	4, f	1, e		
a, f, e, d	12, a	4, f			
a, f, e, d, c	12, a				
a, f, e, d, c, b	12, a	4, f	1, e	2, f	3, a

Dưới đây là cài đặt thuật toán Prim bằng mã giả:

```

void Prim()
{
    F =  $\emptyset$ ;  $V_T = \{u\}$ ;
    while ( $|F| < n-1$ )
    {
        Chọn  $e = \{ \min w(u,v) \ (u \in V_T) \ \& \ (v \notin V_T) \}$ ;
        F = F  $\cup$  {e};
         $V_T = V_T \cup \{v\}$ ;
    }
}

```

} }

- **Nhận xét**

- Thuật toán Kruskal và Prim đều là dạng thuật toán tham lam. Thuật toán tham lam là thuật toán thực hiện một lựa chọn tối ưu ở mỗi giai đoạn. Việc tối ưu hóa ở mỗi giai đoạn của thuật toán không đảm bảo tạo ra lời giải tối ưu toàn cục. *Nhưng hai thuật toán này đều là các thuật toán tham lam tạo các lời giải tối ưu toàn cục.*

- Thuật toán Kruskal làm việc kém hiệu quả với các đồ thị dày ($m \geq n(n-1)/2$, m là số cạnh, n là số đỉnh đồ thị).

- Từ bài toán tìm cây khung nhỏ nhất, ta có thể chuyển qua bài toán tìm cây khung lớn nhất bằng cách đổi dấu các trọng số, sau đó áp dụng các thuật toán tìm cây khung nhỏ nhất.

8.4. CÂY CÓ GỐC

8.4.1. Các định nghĩa

Cho T là một cây với một đỉnh v_0 được chọn làm gốc.

x, y, z là các đỉnh trong T .

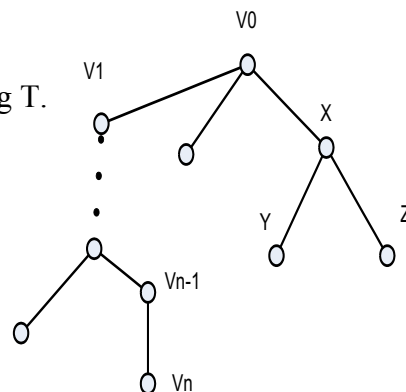
$(v_0, v_1, \dots, v_n)$ là một đường đi đơn trong T .

- **Ta định nghĩa các khái niệm sau:**

- Đỉnh v_{n-1} là **cha** của v_n .

- v_n là **con** của v_{n-1} .

- $v_0, v_1, \dots, v_{n-1}$ là các **tiền bối** của v_n .



Hình 8.7

- Nếu x là tiền bối của y thì y là **hậu duệ** của x .

- Nếu y, z cùng là con của x thì y, z là **anh em** của nhau.

- Nếu x không có con thì x là **lá**. Ngược lại x là **đỉnh trong**.

- **Mức** của đỉnh x là chiều dài (số cạnh) của đường đi đơn từ gốc v_0 tới x .

- Như vậy, mức của v_0 là: $\text{level}(v_0) = 0$.

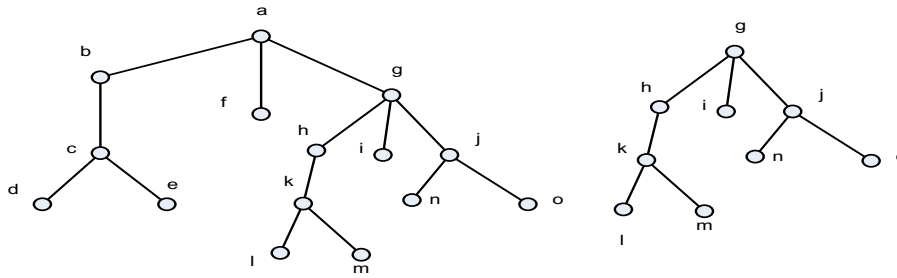
- **Chiều cao** của một cây là mức lớn nhất của các đỉnh trong cây.

- **Cây con** của T gốc tại x là đồ thị con của T mà:

- + Tập đỉnh gồm x và các hậu duệ của x .

- + Tập cạnh gồm mọi cạnh trong T nối từ x tới các hậu duệ của x .

Ví dụ:



Hình 8.8

Cha của c là b

Con của g là h, i, j

Các **tiền bối** của e là c, b, a

Các **hậu duệ** của b là c, d, e

Các **đỉnh trong**: a, b, c, g, h, j, k

Các **lá**: d, e, f, l, m, i, n, o

Mức của c là 2, của k là 3

Chiều cao của cây là 4

Cây con gốc g.

• **Một cây có gốc gọi là:**

▪ **m-cây**: nếu mỗi đỉnh trong có không quá m con.

▪ **m-cây đầy**: nếu mỗi đỉnh trong có đúng m con.

▪ **Cây nhị phân**: nếu mỗi đỉnh trong không quá 2 con.

▪ **Cây có gốc thứ tự**: nếu các con của mỗi đỉnh trong được xếp thứ tự (từ trái qua phải).

▪ Với **Cây nhị phân có thứ tự**: nếu mỗi đỉnh trong đủ 2 con thì:

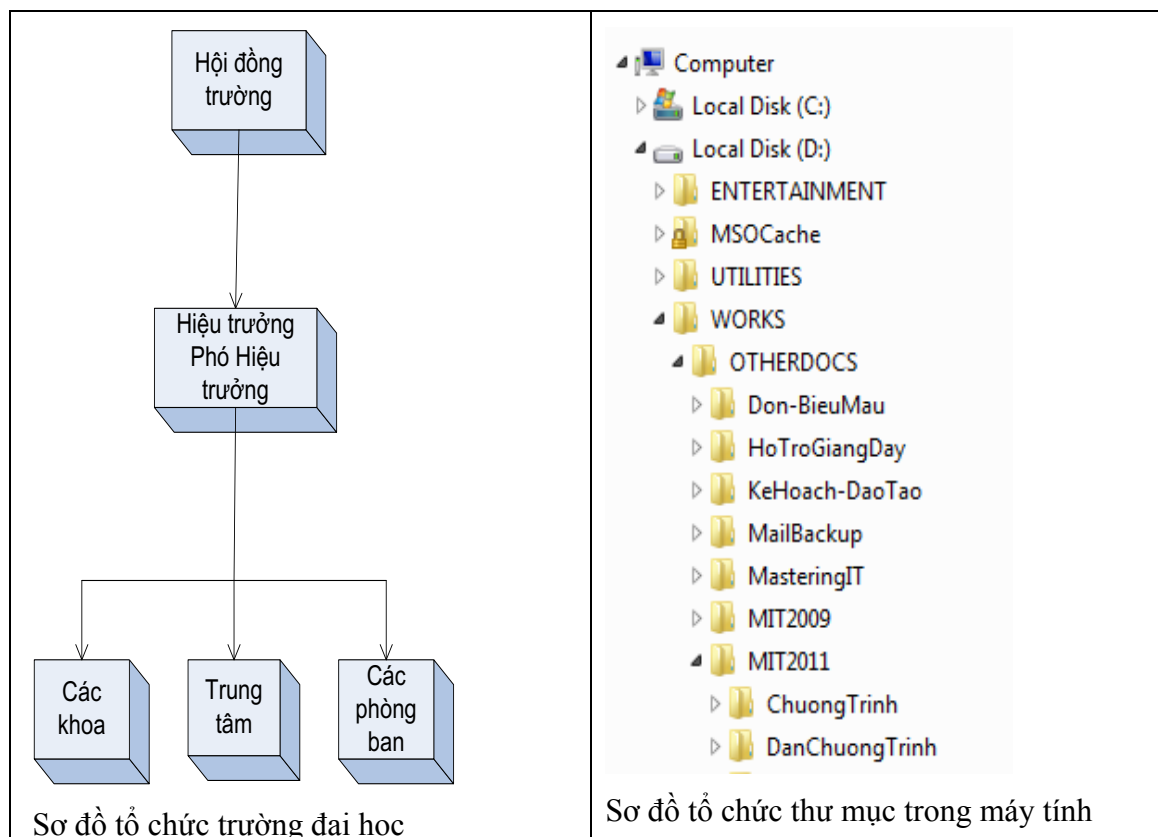
+ Con thứ nhất là **con bên trái**.

+ Con thứ hai là **con bên phải**.

▪ Một m-cây với chiều cao h được gọi là **cây thăng bằng (Cây cân bằng)** nếu tất cả các lá đều ở mức h hay h-1.

• **Một số ví dụ về cây có gốc**

Trong thực tế, có nhiều mô hình sử dụng cây có gốc như: mô hình gia phả trong một dòng họ; mô hình tổ chức của một cơ quan; mô hình các thư mục, tập tin trong máy tính.



Hình 8.9: Ví dụ cây có gốc

8.4.2. Cây nhị phân tìm kiếm

Định nghĩa

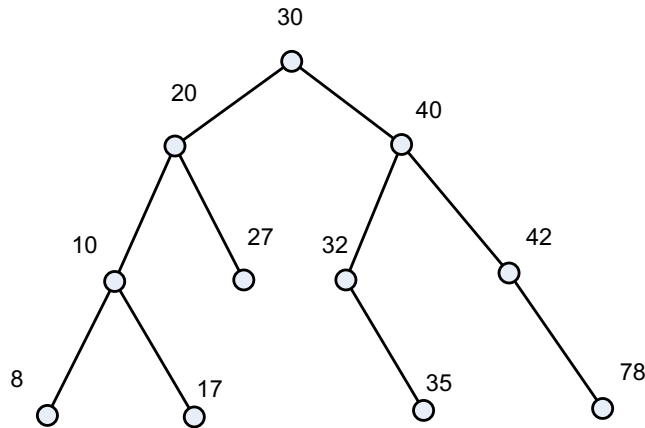
Một cây nhị phân tìm kiếm là một cây nhị phân T trong đó mỗi đỉnh được gán cho một nhãn, các nhãn của đỉnh có thể so sánh được với nhau và với mọi đỉnh $v \in T$, các nhãn trong cây con bên trái của v đều nhỏ hơn nhãn của v và các nhãn trong cây con bên phải của v đều lớn hơn nhãn của v .

Ví dụ:

Tạo cây nhị phân tìm kiếm cho các phần tử có nhãn như sau: 30, 20, 10, 40, 32, 27, 17, 8, 42, 78, 35.

Gốc của cây là phần tử có nhãn 30.

Ta có cây nhị phân tìm kiếm:



Hình 8.10

Thuật toán tìm kiếm trên cây nhị phân tìm kiếm

```

void TK(Cây NPTK T, phần tử x); // Tìm x trên cây NPTK T
{
    v = gốc của T;
    ketqua = "Không tìm thấy x";
    while ((v != NULL) && (label(v) != x))
    {
        if (x == label(v)) ketqua = "Tìm được x";
        if (x < label(v))
            if (con bên trái v != NULL) v = con bên trái v;
        if (x > label(v))
            if (con bên phải v != NULL) v = con bên phải v;
    }
    cout << ketqua;
}
  
```

8.4.3. Cây quyết định

Định nghĩa

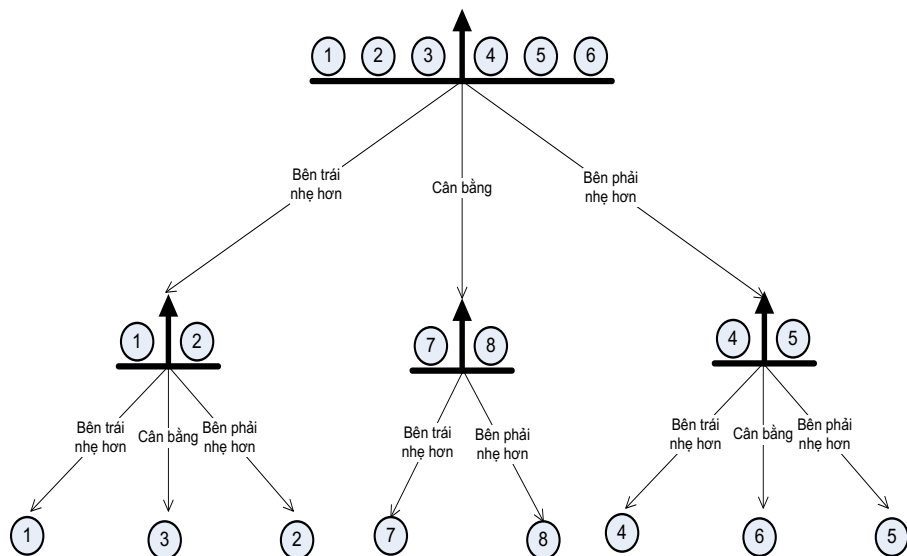
Cây quyết định là cây có gốc trong đó mỗi đỉnh tương ứng với một quyết định và mỗi cây con tại các đỉnh này ứng với mỗi kết cục có thể của quyết định. Một lời giải là một đường đi từ gốc đến lá.

• Ví dụ

Bài toán 8 đồng xu: Cho 8 đồng xu nhìn bề ngoài rất giống nhau, trong đó có 7 đồng xu thật nặng như nhau, và một đồng xu giả nhẹ hơn 7 đồng còn lại. Bằng một cái cân thăng bằng, hãy xác định đồng xu giả.

Giải:

Ta xây dựng cây quyết định từ ngữ cảnh của bài toán như sau: Mỗi lần cân sẽ có 3 trạng thái: Bên trái nặng hơn, bên phải nặng hơn, và thăng bằng. Vậy cây quyết định ta xây dựng là cây tam phân. Cây có ít nhất 8 lá vì có 8 kết cục có thể có và mỗi quyết định cần biểu diễn bằng 1 lá. Khi đó, số lần cân nhiều nhất để xác định đồng xu giả là chiều cao h của cây. Ta có $h \geq \lceil \log_3 8 \rceil = 2$. Sau đây là một cách cân và cây quyết định tương ứng:



Hình 8.11: Bài toán 8 đồng xu

8.4.4. Các phương pháp duyệt cây

Thuật toán viếng thăm mọi đỉnh của một cây có gốc có thứ tự đúng một lần một cách có hệ thống gọi là thuật toán duyệt cây. Dưới đây sẽ giới thiệu ba thuật toán được sử dụng thường xuyên nhất: Duyệt tiền tự (Preorder traversal), Duyệt trung tự (Inorder traversal), Duyệt hậu tự (Postorder traversal).

Thuật toán duyệt tiền tự

Kiểu duyệt này trước tiên là thăm nút gốc, sau đó thăm các nút của các cây con từ trái qua phải.

```
void Preorder(cây thứ tự có gốc T);  
{  
    r = gốc của T;  
    Thăm r;  
    for (Mỗi cây con c của r từ trái sang phải)  
    {  
        T(c) = Cây con với gốc c;  
        Preorder(T(c));  
    }  
}
```

```
}  
}
```

Thuật toán duyệt trung tự

Kiểu duyệt này trước tiên là thăm các nút của cây con bên trái nhất, sau đó thăm nút gốc rồi đến các nút của các cây con còn lại theo thứ tự từ trái qua phải.

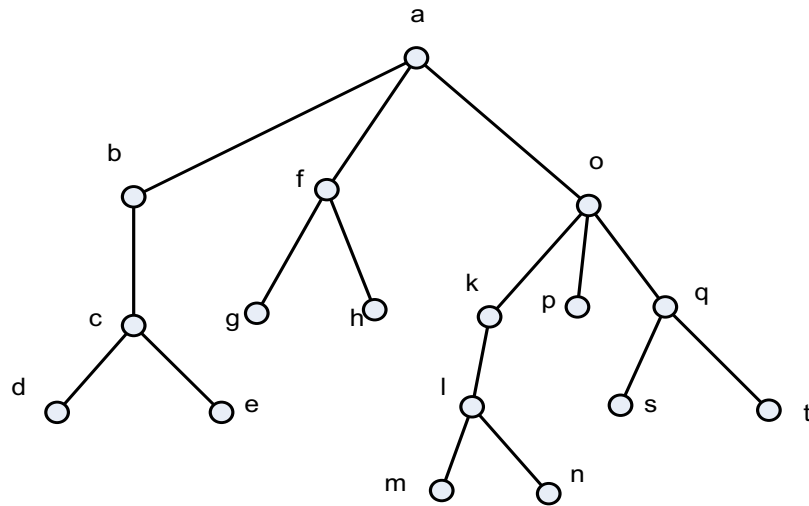
```
void Inorder(cây thứ tự có gốc T)  
{  
    r = gốc của T;  
    if (r là lá) Thăm r;  
    else  
    {  
        s = con đầu tiên từ trái sang phải của r;  
        T(s) = Cây con với gốc s;  
        Inorder(T(s));  
        Thăm r;  
        for (Mỗi cây con c của r từ trái sang phải trừ s)  
            T(c) = Cây con với gốc c;  
            Inorder(T(c));  
    }  
}
```

Thuật toán duyệt hậu tự

Kiểu duyệt này trước tiên là thăm các nút của các cây con từ trái qua phải. Cuối cùng là thăm nút gốc.

```
void Postorder(cây thứ tự có gốc T);  
{  
    r = gốc của T;  
    for (Mỗi cây con c của r từ trái sang phải)  
    {  
        T(c) = Cây con với gốc c;  
        Postorder(T(c));  
    }  
    Thăm r;  
}
```


Ví dụ:



Hình 8.12: Các phương pháp duyệt cây

Thực hiện các phương pháp duyệt khác nhau:

- + Duyệt tiền tự: a, b, c, d, e, f, g, h, o, k, l, m, n, p, q, s, t
- + Duyệt trung tự: d, c, e, b, a, g, f, h, m, l, n, k, o, p, s, q, t
- + Duyệt hậu tự: d, e, c, b, g, h, f, m, n, l, k, p, s, t, q, o, a

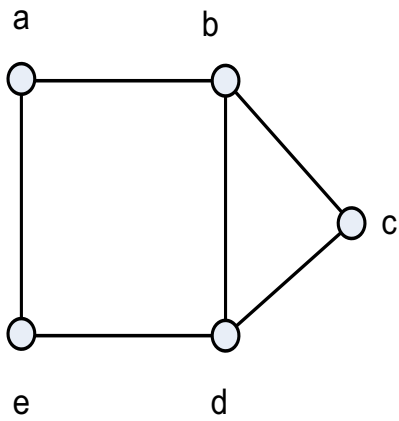
Chú ý:

Nếu cho phép duyệt các cây con từ phải sang trái ta sẽ nhận được thêm 3 thuật toán duyệt cây nữa:

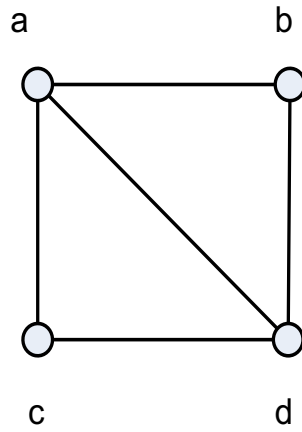
- + Duyệt tiền tự: a, o, q, t, s, p, k, l, n, m, f, h, g, b, c, e, d
- + Duyệt trung tự: t, q, s, o, p, n, l, m, k, a, h, f, g, e, c, d, b
- + Duyệt hậu tự: t, s, q, p, n, m, l, k, o, h, g, f, e, d, c, b, a

8.5. BÀI TẬP CHƯƠNG 8

1. Cho T là một cây 5-phân đầy đủ có 156 đỉnh. Hỏi T có bao nhiêu lá?
2. Cho G là đồ thị đơn vô hướng liên thông có n đỉnh và m cạnh. Hỏi cần phải xóa đi bao nhiêu cạnh trong G để G trở thành 1 cây?
3. Khi nào thì một cạnh của đồ thị G luôn luôn nằm trong tất cả các cây khung của nó ?
4. Hãy vẽ tất cả các cây khung của đồ thị sau

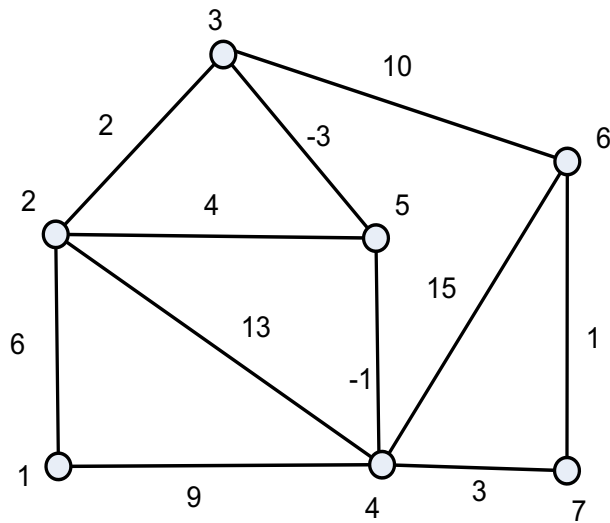


Hình 5.13



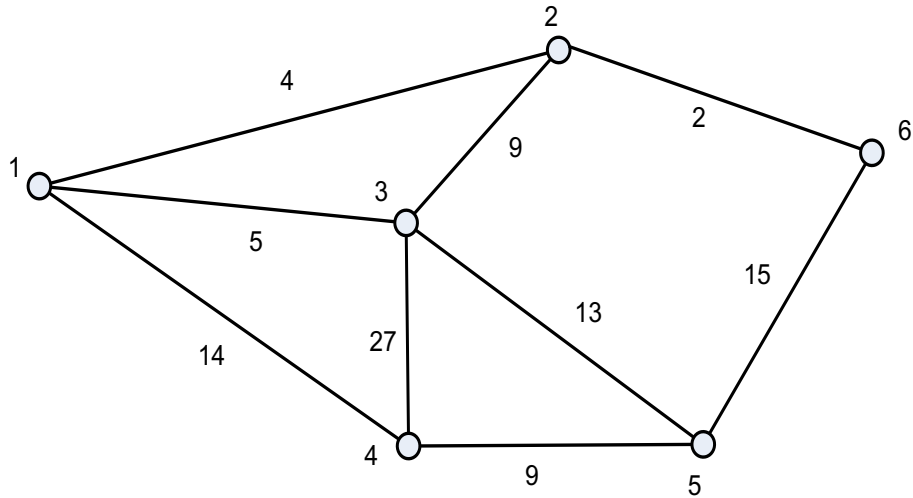
Hình 5.14

5. Áp dụng thuật toán Kruskal, tìm cây khung nhỏ nhất cho đồ thị sau



Hình 5.15

6. Áp dụng thuật toán Prim tìm cây khung nhỏ nhất cho đồ thị sau. Đỉnh chọn đầu tiên là đỉnh 3.



Hình 5.16

7. Cho ma trận kề của một đồ thị vô hướng. Hãy viết chương trình kiểm tra xem nó có phải là cây hay không.

8. Cho ma trận kề của một cây có gốc và một đỉnh của cây. Hãy viết chương trình tìm chiều cao của cây.

9. Cho danh sách các phần tử là các số nguyên. Hãy viết chương trình xây dựng cây tìm kiếm nhị phân chứa các phần tử này.

10. Cho một đồ thị vô hướng được biểu diễn dưới dạng ma trận kề. Hãy viết chương trình cho phép tìm cây khung nhỏ nhất của đồ thị.

a. Sử dụng thuật toán Kruskal

b. Sử dụng thuật toán Prim